

Security Audit

Orlyn (dApp)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	27
Conclusion	28
Our Methodology	29
Disclaimers	31
About Hashlock	32

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Orlyn team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Orlyn creates anonymous, user-owned connections between people and brands using Pocket Crumbs - a new blockchain method for verifying relationships without sharing data.

Orlyn is a decentralized platform that empowers users to take control of their personal data and be rewarded for sharing it with brands. By leveraging blockchain technology, Orlyn ensures that users' data is stored securely and anonymously, allowing them to engage with brands without compromising their privacy.

Orlyn's mission is to take the world's data back from Big Tech and put it in the hands of everyday people, onboarding hundreds of millions into Web3.

Project Name: Orlyn

Project Type: dApp, Token

Compiler Version: ^0.8.42

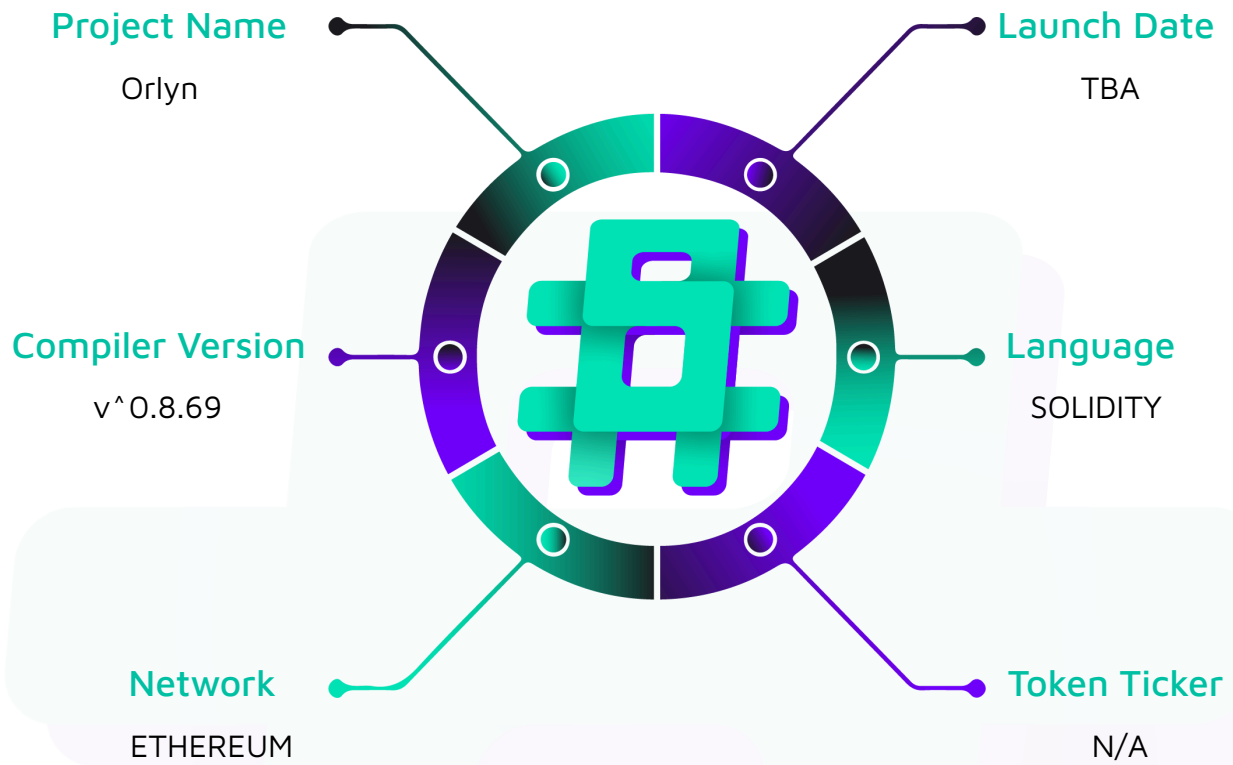
Website: <https://www.orlyn.io/>

Logo:

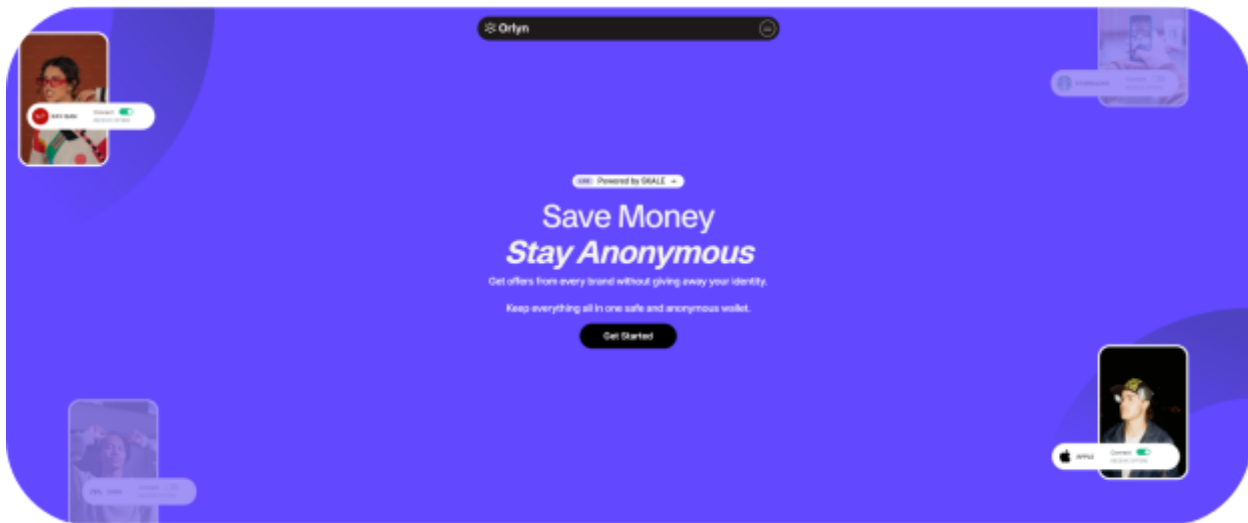


#hashlock.

Hashlock Pty Ltd

Visualised Context:

Project Visuals:



Orlyn

Connect with brands without giving anything away.

Orlyn lets you connect privately to your favourite brands and receive offers, updates, and digital experiences.

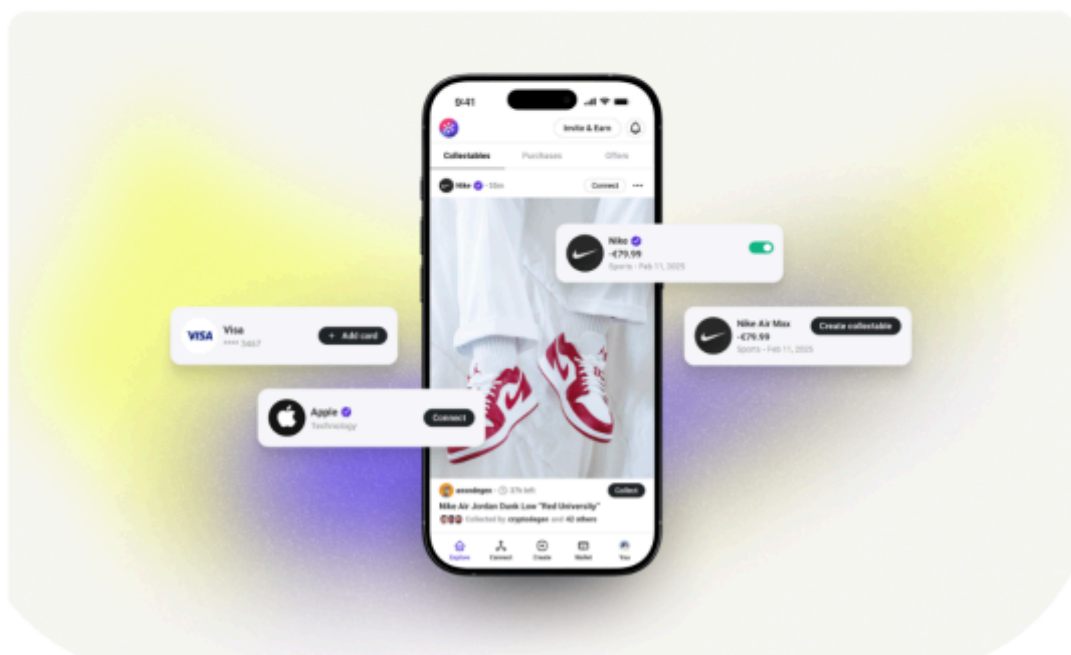
No emails, cookies, or phone numbers.

Your data is owned and stored by you, and if you choose to share it, it's shared anonymously.

Everything stays in your control, all in one wallet.

Get offers from every brand without giving away your identity.

Launch App



#hashlock.

Hashlock Pty Ltd

Audit Scope

We at Hashlock audited the solidity code within the Wal8 project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Orlyn Smart Contracts
Platform	Ethereum / Solidity
Audit Date	April, 2025
Contract 1	Wal8PurchaseRecord.sol
Contract 2	Wal8Collectable.sol
Audited GitHub Commit Hash	0f27beaced1c82a06e3dc4627e222adc50129171
Fix Review GitHub Commit Hash	696241bf16284c7d63b9e9cda6be8d1f574153cd

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerabilities

1 Medium severity vulnerability

5 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Wal8PurchaseRecord.sol Allows users to: - Mint NFTs representing purchase records - Mint with signatures from both user and Wal8 - Store purchase records with merkle roots - View purchase record metadata via CID - Non-transferable tokens except during recovery - Allows admins to: - Batch mint purchase records - Recover tokens to new addresses - Grant minter roles	- Minting works but has signature verification issues due to incorrect EIP-712 encoding - - CID not included in signed data, allowing potential content manipulation - - No validation of merkle roots beyond empty check - - CID validation is insufficient (no format checking) - - Transfer restrictions work as intended - - Vulnerable to DOS attacks through EIP-7702 - Recovery functionality works as intended - Role management works as intended

Wal8Collectable.sol Allows users to: - Create collectibles from purchase records - Collect tokens during collection periods - View token metadata Allows admins to: - Set global parameters - Configure payment tokens - Manage treasury	- No verification of purchase record ownership or existence - URI function missing existence checks - Payment system works as intended - Treasury management works as intended
--	--

Code Quality

This audit scope involves the smart contracts of the Wal8 project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Orlyn project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Wal8PurchaseRecord#batchMint - batchMint Function Griefing Through EIP-7702 Delegation

Description

The batch minting mechanism is vulnerable to a denial-of-service attack through the upcoming EIP-7702's account abstraction.

Vulnerability Details

The upcoming [EIP-7702](#) allows an EOA to delegate its address to a contract, which will make the EOA have code when queried. The delegated address can anytime revoke the delegation and make their EOA code zero again.

A malicious actor could get their EOA approved for minting through an off-chain backend, seeming to be innocent as EOA, then delegate their account to a contract that doesn't implement `onERC721Received` as they monitor the mempool and frontrun the `batchMint` function.

When included in a batch mint operation, this would cause the entire batch to fail, blocking legitimate mints. This vulnerability could be exploited for griefing attacks against the minting process repeatedly.

```
function batchMint(
    address[] memory recipients,
    string[] memory cids,
    uint256[] memory versions,
    string[] memory internalIds,
    string[] memory dates,
    bytes32[] memory merkleRootPurchaseRecords
) public virtual onlyRole(MINTER_ROLE) returns (uint256[] memory) {
    uint256 length = recipients.length;
    require(length > 0 && length <= 500, "Invalid batch size");
```

```

require(
    length == cids.length &&
    length == versions.length &&
    length == internalIds.length &&
    length == dates.length &&
    length == merkleRootPurchaseRecords.length,
    "Array lengths must match"
);

uint256[] memory tokenIds = new uint256[](length);

for (uint256 i = 0; i < length; i++) {
    tokenIds[i] = mint(
        recipients[i], <<
        cids[i],
        versions[i],
        internalIds[i],
        dates[i],
        merkleRootPurchaseRecords[i]
    );
}
return tokenIds;
}

```

Impact

This would cause the entire batch to fail, blocking legitimate mints. This vulnerability could be exploited for griefing attacks against the minting process repeatedly.

Recommendation

Implement a try/catch pattern in the batchMint function to handle individual mint failures gracefully without reverting the entire batch.

```

function batchMint(
    address[] memory recipients,
    string[] memory cids,
    uint256[] memory versions,
    string[] memory internalIds,

```

```

    string[] memory dates,
    bytes32[] memory merkleRootPurchaseRecords
) public virtual onlyRole(MINTER_ROLE) returns (uint256[] memory) {
    uint256 length = recipients.length;
    require(length > 0 && length <= 500, "Invalid batch size");
    require(
        length == cids.length &&
        length == versions.length &&
        length == internalIds.length &&
        length == dates.length &&
        length == merkleRootPurchaseRecords.length,
        "Array lengths must match"
    );
    uint256[] memory tokenIds = new uint256[](length);
    for (uint256 i = 0; i < length; i++) {
        try this.mint(
            recipients[i],
            cids[i],
            versions[i],
            internalIds[i],
            dates[i],
            merkleRootPurchaseRecords[i]
        ) returns (uint256 tokenId) {
            tokenIds[i] = tokenId;
        } catch Error(string memory reason) {
            emit MintFailed(recipients[i], reason);
        }
    }
    return tokenIds;
}

```

Status

Resolved

[H-02] Wal8PurchaseRecord#mintWithSignature - CID is Not Part Of The Signed Data Structure

Description

The CID parameter, which points to the purchase data on IPFS, is passed separately from the signed PurchaseRecord struct. This creates a vulnerability where an attacker can frontrun the mintWithSignature function with the valid signature and mint the NFT with a different CID.

Vulnerability Details

Since the CID isn't part of what's being signed, there's no cryptographic link between the purchase record data and its IPFS content. This allows for potential CID injection while maintaining valid signatures. The attacker can frontrun the function being called and inject their CID, which breaks the intended mechanism at all

```
function mintWithSignature(  
    string memory cid,  
    PurchaseRecord memory purchaseRecord,  
    PurchaseRecordSignature memory purchaseRecordSignatureRecipient,  
    PurchaseRecordSignature memory purchaseRecordSignatureWal8  
) public virtual nonReentrant returns (uint256) {  
    ...  
}
```

Impact

The minted NFT will refer to a different CID where it will phish the users.

Recommendation

Include the CID as part of the PurchaseRecord struct and update the EIP-712 typehash to include it in the signature verification process.

Status

Resolved

Medium

[M-01] Wal8PurchaseRecord#PURCHASE_RECORD_TYPEHASH - Incorrect
EIP-712 Type Encoding for merkleRootPurchaseRecords

Description

The EIP-712 type encoding for merkleRootPurchaseRecord is incorrectly defined as string (a dynamic type) when the actual struct uses bytes32 (an atomic type).

Vulnerability Details

According to [EIP-712](#)'s type system, dynamic types like string are encoded by first taking their keccak256 hash, while atomic types like bytes32 are used directly in the encoding. This type mismatch means that when wallets sign the data, they'll treat the merkleRoot as a dynamic type and hash it first, while the contract expects it as an atomic type that should be used as-is. This creates fundamentally different encodings:

- As string (current incorrect encoding):

```
keccak256(abi.encode(keccak256(bytes(merkleRoot))))
```

- As bytes32 (correct encoding): keccak256(abi.encode(merkleRoot))

```
bytes32 public constant PURCHASE_RECORD_TYPEHASH = keccak256("PurchaseRecord(uint256  
version,string internalId,string date,string partnerId,string merkleRootPurchaseRecord)"
```

```
^^^^
```

Impact

This mismatch breaks the signature verification process as the signed hash and the on-chain verified hash will not match, even for valid data. The issue fundamentally breaks the EIP-712 signing mechanism for purchase records.

Recommendation

Update the PURCHASE_RECORD_TYPEHASH to correctly encode merkleRootPurchaseRecord as bytes32:

```
bytes32 public constant PURCHASE_RECORD_TYPEHASH = keccak256("PurchaseRecord(uint256  
version,string internalId,string date,string partnerId,bytes32 merkleRootPurchaseRecord)")
```

Status

Resolved



Low

[L-01] Wal8Collectable#createCollectable - No Validation of Purchase Record Reference

Description

The `createCollectable` function accepts any `bytes32` value as a `purchaseRecordRef` without validating its existence or ownership in the `Wal8PurchaseRecord` contract.

Vulnerability Details

Anyone can create collectibles by providing arbitrary `bytes32` values, completely bypassing the purchase verification system. This vulnerability allows unauthorized reward minting and undermines the entire platform's integrity by breaking the connection between purchases and rewards.

```
function createCollectable(
    string memory tokenURI,
    bytes32 purchaseRecordRef <<
) public virtual nonReentrant returns (uint256) {
    Wal8CollectableStorage storage $ = _getStorage();
    uint256 tokenId = ++$.nextTokenId;

    // Mint initial supply to creator
    _mint(msg.sender, tokenId, 1, "");

    // Set token URI and purchase record reference
    _setURI(tokenId, tokenURI);
    $.purchaseRecordRefs[tokenId] = purchaseRecordRef; <<

    emit CollectableCreated(tokenId, msg.sender, purchaseRecordRef, tokenURI);

    return tokenId;
}
```

Impact

This fundamentally breaks the core business logic that requires valid purchases for creating collectibles.

Recommendation

Implement cross-contract validation to verify that the `purchaseRecordRef` exists in the `Wal8PurchaseRecord` contract and the related NFT is owned by the caller.

Status

Resolved

[L-02] `Wal8Collectable#createCollectable` - Multiple ERC1155s from Same Purchase Record

Description

The contract allows the same `purchaseRecordRef` to be used multiple times to create different ERC1155 tokens.

Vulnerability Details

This creates an economic vulnerability where a single purchase can generate unlimited different reward tokens. Each of these tokens can then be collected multiple times during their collection periods, leading to dilution of legitimate collections.

```
function createCollectable(
    string memory tokenURI,
    bytes32 purchaseRecordRef <<
) public virtual nonReentrant returns (uint256) {
    Wal8CollectableStorage storage $ = _getStorage();
    uint256 tokenId = ++$.nextTokenId;
    // Mint initial supply to creator

    _mint(msg.sender, tokenId, 1, "");

    // Set token URI and purchase record reference
    _setURI(tokenId, tokenURI);
```

```

$.purchaseRecordRefs[tokenId] = purchaseRecordRef;

emit CollectableCreated(tokenId, msg.sender, purchaseRecordRef, tokenURI);

return tokenId;
}

```

Impact

This breaks the intended one-to-one relationship between purchases and rewards, potentially destabilizing the entire reward economy.

Recommendation

Implement a mapping to track used purchase records and ensure each can only be used once for creating collectibles.

Status

Resolved

[L-03] Wal8Collectable#uri - Missing Token Existence Check in URI Function

Description

The `uri` function returns values without verifying token existence, creating inconsistency in how non-existent tokens are handled. This could cause confusion for integrating systems and users, as there's no clear distinction between existent and non-existent tokens when querying URIs.

Vulnerability Details

The `uri` function returns the same base URI for both existent and non-existent tokens: For non-existent tokens, it returns the base URI without any validation, the same behavior as valid tokens.

```

function uri(uint256 tokenId) public view virtual override(ERC1155Upgradeable,
ERC1155URIStorageUpgradeable) returns (string memory) {
    return ERC1155URIStorageUpgradeable.uri(tokenId);
}

```

Impact

While not directly exploitable, this deviation from standard practices could lead to integration issues. External systems cannot distinguish between valid and invalid token URIs.

Recommendation

Add a token existence check before returning the URI.

```
function uri(uint256 tokenId) public view virtual override(ERC1155Upgradeable, ERC1155URIStorageUpgradeable) returns (string memory) {  
    require(exists(tokenId), "Token does not exist");  
    return ERC1155URIStorageUpgradeable.uri(tokenId);  
}
```

Status

Resolved

[L-04] Wal8Collectable#supportsInterface - Incomplete Interface Support in supportsInterface

Description

The contract inherits from multiple interfaces but doesn't properly declare support for all of them in the `supportsInterface` function. The current implementation only explicitly checks for the `UUPSUpgradeable` interface and relies on the parent's `'supportsInterface'` for others.

Vulnerability Details

The contract inherits from:

- `ERC1155Upgradeable`
- `ERC1155URIStorageUpgradeable`
- `ERC1155SupplyUpgradeable`
- `OwnableUpgradeable`
- `UUPSUpgradeable`

- ReentrancyGuardUpgradeable

But only explicitly declares support for:

- UUPSUpgradeable

- Base ERC1155 interfaces (through super call)

When external contracts check for interface support using `supportsInterface(interfaceId)`, they may receive incorrect responses for interfaces like `ERC1155URIStorage` or `ERC1155Supply`, potentially causing integration issues.

```
function supportsInterface(bytes4 interfaceId) public
    view
    virtual
    override(ERC1155Upgradeable)
    returns (bool)
{
    return interfaceId == type(UUPSUpgradeable).interfaceId ||
        super.supportsInterface(interfaceId);
}
```

Impact

This oversight could cause issues with interface detection in integrating systems and might break contract interaction checks. While not immediately dangerous, incomplete interface support can lead to integration difficulties and reduced contract interoperability.

Recommendation

Add explicit support checks for all inherited interfaces and ensure proper override implementation.

```
function supportsInterface(bytes4 interfaceId)
    public
    view
    virtual
    override(ERC1155Upgradeable, ERC1155URIStorageUpgradeable)
    returns (bool)
```



```
{
    return interfaceId == type(UUPSUpgradeable).interfaceId ||
        interfaceId == type(IERC1155).interfaceId ||
        interfaceId == type(IERC1155MetadataURI).interfaceId ||
        interfaceId == type(IERC1155Supply).interfaceId ||
        super.supportsInterface(interfaceId);
}
```

Status

Resolved

[L-05] Wal8PurchaseRecord#mint - Insufficient CID Validation

Description

While the contract restricts CID setting to trusted minters through the `MINTER_ROLE`, the validation of CIDs is still overly simplistic, only checking for non-empty strings. Even with trusted minters, this creates potential operational risks through accidental input of invalid CIDs.

Vulnerability Details

The current implementation lacks safeguards against accidental errors in CID formatting. Despite having trusted minters, there's no protection against human error or typos when inputting CIDs. The system accepts any non-empty string as valid, which means malformed CIDs could be stored due to copy/paste errors or script bugs. Furthermore, the lack of standardization enforcement means different minters might use inconsistent CID formats (mixing CIDv0 and CIDv1), potentially complicating metadata handling across the platform.

```
function mint(
    address recipient,
    string memory cid,
    uint256 version,
    string memory internalId,
    string memory date,
    bytes32 merkleRootPurchaseRecord
) public virtual onlyRole(MINTER_ROLE) nonReentrant returns (uint256) {
```

```
require(bytes(cid).length > 0, "CID cannot be empty"); <<  
require(recipient != address(0), "Recipient cannot be zero address");  
// rest of the function
```

Impact

Invalid CIDs could lead to metadata retrieval failures and broken links in NFT marketplaces. This creates a maintenance burden requiring manual intervention to fix incorrect CIDs and potentially impacts user experience when metadata cannot be properly retrieved.

Recommendation

At minimum, implement basic length validation for CIDs since no valid IPFS CID can be shorter than 46 characters:

```
function mint(  
    address recipient,  
    string memory cid,  
    // ... other parameters  
) public virtual onlyRole(MINTER_ROLE) nonReentrant returns (uint256) {  
    require(bytes(cid).length >= 46, "Invalid CID: minimum length is 46 characters");  
    // ... rest of the function  
}
```

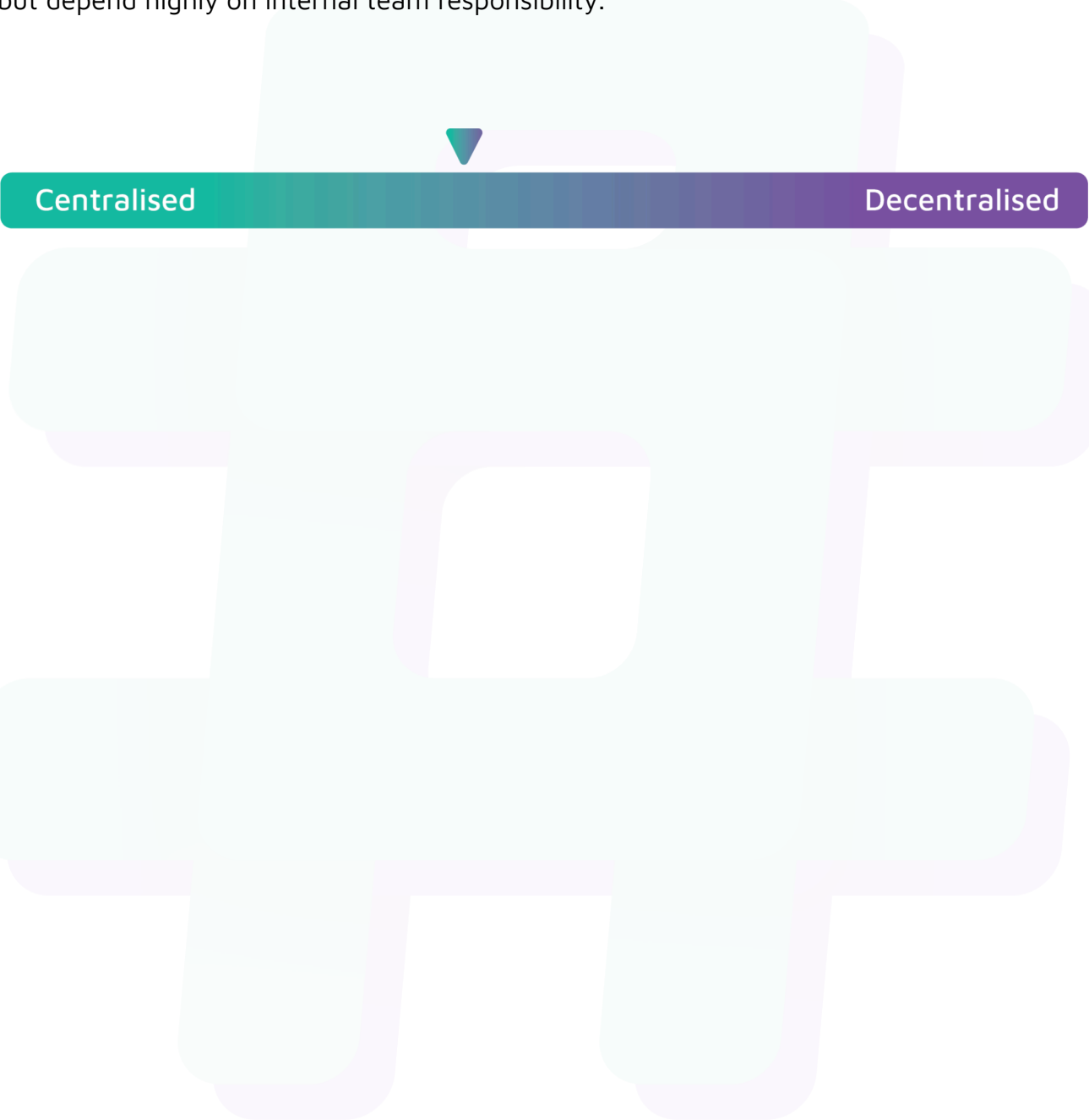
Status

Resolved

Centralisation

The Orlyn project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Orlyn project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.